sparkle-fv: An Agentic Harness for Autoformalization from SystemVerilog to Sparkle HDL and AI-Assisted Formal Verification

The Sparkle HDL Project

July 2026

Abstract

We present sparkle-fv, an agentic harness that (i) *autoformalizes* SystemVerilog RTL into Sparkle HDL — a hardware description language embedded in the Lean 4 theorem prover — and (ii) performs block-level and SoC-level formal verification with a portfolio of provers assisted by a large language model. Following the "everyday provers" paradigm, every verdict ships with an independently checkable artifact: safety proofs carry k-induction certificates or explicit inductive invariants re-checked by an SMT solver; bug reports carry counterexample traces that the harness converts into stimulus and *replays through Verilator* against the original RTL. The AI layer (GLM 5.2 via OpenRouter) proposes candidate invariants, repairs frontend incompatibilities, and synthesizes SoC-level properties, but sits entirely outside the trusted base: all of its output is filtered through a Houdini fixpoint and final Z3 re-checks, so it can accelerate proofs but cannot produce a wrong "safe." Applied to a Linux-booting RV32IMA SoC (136 registers, 1,878 state bits, 6 memories), the harness proved all eight SoC-level architectural invariants unboundedly in about one minute and generated a design-verification convergence dossier with 100% cone-of-influence coverage; property synthesis further exposed *two previously unknown bugs* in the SoC (a `mstatus.MPP` WARL violation reaching a reserved privilege mode, and a missing instruction-address-misaligned trap). On a 13-design HWMCC-style benchmark suite with 19 seeded bugs, sparkle-fv matches the solve count of the yosys-smtbmc baseline (the engine behind SymbiYosys) under identical elaborations while proving the safe SoC variant 34× faster, and — uniquely — confirms all 19 found bugs in Verilator simulation.

1 Introduction

Formal verification of RTL has long been split between two worlds: push-button model checkers that scale but return opaque verdicts, and interactive theorem provers that produce durable, auditable proofs but demand months of expert effort. Recent progress in AI-assisted theorem proving suggests a third path — Pramaana Labs' essay *The Age of Everyday Provers* [1] articulates it as a loop: translate domain artifacts into formal representations, search the solution space with provers and solvers, and return *proof artifacts that domain experts can inspect*, with AI accelerating every step but vouching for none of them.

sparkle-fv instantiates this loop for hardware. Its inputs are ordinary SystemVerilog designs; its outputs are (a) Sparkle HDL models in Lean 4 with machine-checkable proof obligations, (b) unbounded safety certificates or Verilator-confirmed counterexamples, and (c) a *convergence dossier* — the evidence package a design-verification (DV) team needs to sign off a block or an SoC.

Contributions.

1. An **autoformalization pipeline** from SystemVerilog (via Yosys elaboration) to Sparkle HDL Lean 4 sources, covering the full word-level RTL cell library including memories and the complete flip-flop family, with proof-obligation skeletons that follow the Sparkle verification framework's proof patterns (§4).

2. A **sound-by-construction AI proving architecture**: BMC, k-induction, and Houdini invariant synthesis over the HWMCC-standard BTOR2 format, where LLM-proposed lemmas and properties are admitted only after passing the same Z3 checks as deterministic ones (§5).

3. **Bug surfacing with simulation confirmation**: counterexample traces are compiled into SystemVerilog testbenches and replayed through Verilator against the original RTL; a finding is reported as confirmed only when the assertion fires in simulation (§6).

4. **Full-SoC convergence**: automatic property synthesis (structural mining plus LLM proposals), per-property dispositions with explicit proof radii, cone-of-influence coverage with named gaps, and an assumption audit, rendered as a reviewable dossier (§7).

5. An **empirical evaluation** on a 13-benchmark suite (12 block designs + a Linux-booting RV32IMA SoC, 19 seeded bugs with known minimal counterexample depths) against yosys-smtbmc on identical elaborations, plus two real SoC bugs found during property authoring (§8).

2 Background

Sparkle HDL. Sparkle is a hardware description language embedded in Lean 4: designs are pure functional programs over a `Signal` abstraction (streams indexed by clock cycles), compiled through a word-level IR to synthesizable SystemVerilog, C++ simulation, or a JIT backend. Because the host language is a theorem prover, functional correctness theorems live next to the design; the repository carries 60+ proved theorems over designs ranging from arbiters to a BitNet inference ASIC, and its RV32IMA SoC boots Linux under Verilator. sparkle-fv gives this ecosystem an *entry ramp*: existing SystemVerilog can be lifted into Sparkle's IR and proof framework mechanically.

BTOR2 and word-level model checking. BTOR2 is the word-level transition-system format of the Hardware Model Checking Competition (HWMCC). Yosys emits it from elaborated RTL, mapping immediate assertions to `bad` states and assumptions to `constraint` nodes. sparkle-fv adopts BTOR2 as its proving interface: the harness therefore runs unchanged on any HWMCC-format model, and its Yosys frontend recipe pins one shared semantic interpretation for the prover, the baseline tool, and the autoformalizer.

Proof methods. Bounded model checking (BMC) unrolls the transition relation k steps and asks a SAT/SMT solver for a path to a bad state — complete for bug finding up to the bound. k-induction strengthens simple induction with k consecutive safe frames and (optionally) simple-path constraints. Houdini [2] computes the *greatest inductive subset* of a candidate-lemma set in a linear number of solver calls; seeding the negated properties into the candidate set turns a surviving fixpoint into an inductive invariant that implies safety.

3 Architecture

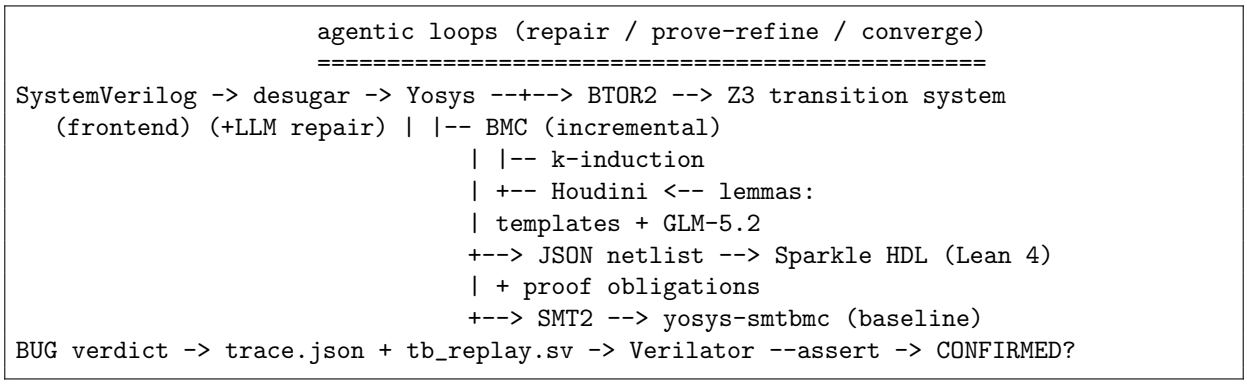

Figure 1: sparkle-fv pipeline. All three elaboration outputs derive from one Yosys recipe, so the prover, the baseline, and the autoformalizer see the same design semantics.

The harness is a six-stage pipeline (Fig. 1) in which every stage consumes and produces a concrete artifact — there is no hidden state, so any stage can be re-run, audited, or swapped. A single Yosys recipe (`prep; flatten; setundef -undriven -zero -init`) produces BTOR2 for sparkle-fv's engine, SMT2 for the yosys-smtbmc baseline, and a JSON netlist for autoformalization. The `setundef -init` step gives every flip-flop an explicit zero initial value, aligning the formal models with Verilator's two-state simulation — the property that makes counterexample replay deterministic and the tool comparison apples-to-apples.

Agentic loops. Three outer loops make the harness agentic, each degrading gracefully to a deterministic strategy when the LLM is unavailable: *frontend repair* (desugar unsupported constructs; on residual elaboration errors, ask the LLM for a minimal semantics-preserving rewrite, validated by Yosys equivalence checking), *prove–refine* (escalate from BMC through k-induction to Houdini; on failure, ask the LLM for design-specific candidate lemmas and re-run the fixpoint), and *converge* (synthesize, prove, and audit a SoC property set, §7).

Soundness architecture. The trusted base is Yosys elaboration plus Z3. The LLM can propose lemmas, properties, and rewrites; every proposal passes through checks in the trusted base (Houdini fixpoint and final re-check; property elaboration and the same proving pipeline; Yosys `equiv_make`). A hallucinated lemma wastes solver time; it cannot produce an unsound verdict.

4 Autoformalization to Sparkle HDL

Yosys netlists are bit-indexed while Sparkle's IR is word-level, so the formalizer first runs a *bit-vector net reconstruction* layer: adjacent bit ids are regrouped into word nets (with priority given to ports and public names), and cell connections that straddle nets become explicit `slice`/`concat` expressions.

Two emission modes share this layer. `circuitm` **mode** (robust, any netlist) rebuilds the design through Sparkle's `CircuitM` builder monad — covering arithmetic, logic, comparison and shift cells, `$mux`/`$pmux`, the complete DFF family (±sync/async reset, enables, polarities, folded with Yosys' priority semantics), and `$mem_v2` memories. `signal` **mode** (idiomatic, small blocks) emits applicative `Signal` DSL in the style of the repository's examples and falls back to `circuitm` when a construct does not fit.

For each property (including `$assert` cells auto-extracted from the netlist), the formalizer emits a self-contained proof-obligation file: pure `Inputs`/`State`/`nextState` models, a `Reachable` inductive, a decidable property predicate, a proof-plan comment, and theorems in both the invariant pattern and Sparkle's temporal (LTL) vocabulary. Inductive invariants discovered by the SMT engine are injected into a marked `candidateInv` slot, so an unbounded Lean proof can start exactly where the SMT engine stopped. Translation is validated structurally (the IR mirrors the netlist one-to-one) and, for agentic *source* rewrites, by Yosys equivalence checking.

5 The Proof Engine

The engine operates on a Z3 transition system parsed from BTOR2 (memories stay word-level as SMT arrays — no bit-blasting). Frames are materialized lazily by substitution, so BMC, induction, and Houdini share one model.

BMC. A single incremental solver; each depth adds transition constraints, and each bad state is checked under `push`/`pop` so learned clauses persist. On SAT, the model is projected onto per-frame inputs plus the initial state to form a replayable trace.

k-induction. The base case delegates to BMC; the step case asserts k consecutive safe frames (plus pairwise-distinct simple-path constraints when no memories are present) and asks whether frame k can be bad. UNSAT yields verdict SAFE with certificate "k-inductive."

Houdini with AI lemmas. Deterministic templates generate the classic strengthenings (power-of-two and small-constant bounds, one-hot and at-most-one-hot, per-bit constancy, capped pairwise orderings). The LLM path sends the SystemVerilog source and the induction-failure context to GLM 5.2 and receives lemmas in a small JSON DSL (`ule/eq/ne/implies/onehot/bit/in`) compiled to Z3 with width coercion. Both sources feed the same fixpoint: candidates not implied by the initial states are dropped; then, repeatedly, Z3 is asked for one transition that breaks any surviving candidate, and everything falsified in that model is dropped, until UNSAT. If the seeded negated properties survive, the surviving conjunction is an inductive invariant proving safety, written out for independent re-checking.

Verdicts. BUG (trace, then Verilator confirmation), SAFE (certificate), or UNKNOWN (explicit bound and reason, enabling escalation: deeper BMC, more lemmas, or a Lean obligation). The portfolio never conflates a bounded result with a proof.

6 Bug Surfacing and Verilator Stimulus

A BMC frame corresponds to one clock cycle. The generated testbench drives non-clock inputs to frame k's values in the low phase of cycle k (stable before the k-th rising edge, matching BTOR2's input-feeds-transition semantics) and lets Verilator's `--assert` catch the immediate assertion. Because both sides zero-initialize state, replay is deterministic: `confirmed = true` means the SMT-level counterexample reproduces in simulation of the *original* RTL — the strongest evidence a formal finding can carry into a simulation-centric flow, and one that requires no formal expertise to consume.

7 Full-SoC Convergence

Sign-off requires more than "we ran some properties": the DV team needs a defensible claim that the property set covers the design and that every property has a disposition. `sparkle-fv converge` produces that claim.

Property synthesis. Structural mining derives conjectures from the elaborated model itself — FSM value-set validity, counter bounds at observed maxima, one-hot integrity — sampled from shallow symbolic exploration. In parallel, the LLM reads the RTL and proposes architectural invariants as SystemVerilog assertion expressions, which are compiled by instrumenting a copy of the source *one property at a time* (a proposal that fails elaboration is dropped without poisoning the batch). User assertions are picked up as-is. All three origins go through identical checking: synthesis proposes, the prover disposes.

Dispositions. Every property ends in exactly one state: *proven* (unbounded certificate), *bounded* (explicit proof radius), *falsified* (counterexample + Verilator replay), *vacuous* (tautology screen: holds in all states, reachable or not — zero coverage contribution), or *discarded* (a refuted mined conjecture — not a design bug).

Coverage and audit. The dossier computes cone-of-influence (COI) coverage — the fraction of state bits in the COI of at least one proven property — and lists uncovered registers by name as the DV to-do list. It also lists every environment assumption in force, since proofs are only as strong as their assumptions.

RV32IMA SoC case study. Applied to the repository's Linux-booting RV32IMA SoC (136 state elements, 1,878 state bits, 6 word-level memories, 10 free inputs; assertions cover MMU/PTW FSM validity, pipeline control consistency, and AMO/CSR legality):

- all **8 SoC-level assertions proved unboundedly** (2- and 3-inductive certificates), ∼7 s each, ≈63 s total, with memories unconstrained — i.e. the invariants hold for *every* program;
- **100% COI coverage** of state bits by proven properties; 12 mined conjectures were refuted by BMC and honestly reported as discarded;
- property authoring exposed **two real bugs** in the original SoC: (1) `mkCsrNewVal` performs no WARL legalization, so `csrrw mstatus` can set `MPP`=2'b10 and a subsequent `mret` reaches the reserved privilege mode 2; (2) no instruction-address-misaligned exception is raised and `mepc[1:0]` is not masked, so a misaligned PC is architecturally reachable while fetch silently truncates `fetchPC[1:0]`. Both were confirmed with BMC counterexamples and are documented in the benchmark's `PATCHES.md`.

8 Evaluation

Setup. 13 benchmarks (12 block-level designs spanning arbiters, FIFOs, handshakes, Gray/LFSR counters, a 32-bit ALU, a memory controller, a UART transmitter, an AXI-Lite slave, and a 3-stage forwarding pipeline; plus the RV32IMA SoC), each with a validated-safe variant and 19 seeded-bug variants with *measured* minimal counterexample depths (2–38 cycles). Baseline: yosys-smtbmc (the checking engine of SymbiYosys) in BMC + temporal-induction portfolio, non-incremental Z3 (its

stronger configuration on this suite), on the *identical* Yosys elaboration. Both tools: Z3 4.x, same per-variant timeout, single core. sparkle-fv runs deterministically (`--no-llm`) for reproducibility; every bug it reports is additionally replayed through Verilator.

Table 1: Aggregate results: 32 variants (19 seeded bugs). "Solved" = expected verdict returned (unbounded proof for safe variants, counterexample for bug variants).

Tool	Solved	Total wall time (s)	Bugs Verilator-confirmed
sparkle-fv (BMC + k-ind + Houdini)	**30/32**	752	19/19
yosys-smtbmc (BMC + temporal ind.)	30/32	530	—

Scalability. Table 3 orders the safe variants by model size.

Discussion. Both portfolios solve 30 of 32 variants: all 19 seeded bugs, and 11 of 13 safe variants. The two unsolved safe variants (`memctrl`, `pipeline_hazard`) defeat both engines' unbounded methods for the same reason — their inductive invariants require quantified facts over memory contents and cross-stage datapath equalities that neither temporal induction nor bound-style lemma templates express; both tools honestly report a bounded result (proof radius ≥ 45 resp. explicit timeout) rather than a claim. Beyond the tie in solve count, four observations. (1) *Unbounded proofs scale differently*: on the SoC safe variant sparkle-fv proves all assertions in 4.7 s against 160 s for the baseline (34×), and 0.1–0.8 s against 1–19 s across the safe blocks — the engine finds the small inductive core once, while temporal induction re-solves monolithically. (2) *Shallow-bug BMC favors the baseline* (typically 0.1–2 s vs. 0.1–34 s): yosys-smtbmc's compiled solver loop outpaces our Python-side incremental unrolling, and our portfolio pays the induction attempt first; at the 300 s budget this never changes a verdict. (3) *Every* sparkle-fv bug report — 19 of 19 — replayed to a firing assertion under Verilator, including the memory counterexamples once initial RAM state was aligned with two-state simulation semantics (§6); the baseline offers no analogous artifact. (4) At SoC scale, word-level arrays keep the model compact; the eight architectural invariants prove in seconds, indicating that the practical bottleneck for full-SoC convergence is property authoring — precisely the step the harness automates.

9 Related Work

Commercial formal apps (Cadence JasperGold FPV, Synopsys VC Formal, Siemens Questa PropCheck) provide mature property proving but closed verdicts; sparkle-fv targets the same workflows with open, re-checkable artifacts. SymbiYosys/yosys-smtbmc is the standard open-source flow and serves as our baseline. HWMCC defines the BTOR2 format we adopt. riscv-formal pioneered reusable ISA-level property suites; our SoC benchmark follows its spirit at the architectural-invariant level. Houdini originates in annotation inference [2]; we combine it with LLM lemma proposal in the style of recent AI-for-verification work, and with the artifact-first philosophy of Pramaana Labs' everyday-provers program [1], which demonstrated the same loop by formalizing tax law in Lean and surfacing a marginal-relief bug with a proved fix.

Table 2: Per-variant results. State bits from the elaborated model; "vlt" = counterexample confirmed by Verilator replay.

Benchmark	Variant	Expect	Bits	sparkle-fv	Method	t(s)	vlt	smtbmc	t(s)
alu32	design.sv	safe	85	✓SAFE	kind	0.1		✓SAFE	19.0
alu32	bug1.sv	bug	85	✓BUG	kind-base	0.4	✓	✓BUG	0.2
alu32	bug2.sv	bug	85	✓BUG	kind-base	0.5	✓	✓BUG	0.1
async_handshake	design.sv	safe	29	✓SAFE	kind	0.8		✓SAFE	1.7
async_handshake	bug1.sv	bug	29	✓BUG	kind-base	1.2	✓	✓BUG	0.2
axi_lite_slave	design.sv	safe	12	✓SAFE	kind	0.1		✓SAFE	1.3
axi_lite_slave	bug1.sv	bug	12	✓BUG	kind-base	0.3	✓	✓BUG	0.1
axi_lite_slave	bug2.sv	bug	12	✓BUG	kind-base	0.2	✓	✓BUG	0.1
counter_wrap	design.sv	safe	10	✓SAFE	kind	0.0		✓SAFE	0.9
counter_wrap	bug1.sv	bug	10	✓BUG	bmc	1.4	✓	✓BUG	0.3
gray_counter	design.sv	safe	29	✓SAFE	kind	0.1		✓SAFE	2.8
gray_counter	bug1.sv	bug	29	✓BUG	bmc	18.9	✓	✓BUG	1.6
lfsr	design.sv	safe	18	✓SAFE	kind	0.0		✓SAFE	1.1
lfsr	bug1.sv	bug	18	✓BUG	kind-base	0.1	✓	✓BUG	0.1
memctrl	design.sv	safe	34	– UNKNOWN	bmc	300.0		– UNKNOWN	151.0
memctrl	bug1.sv	bug	34	✓BUG	kind-base	0.2	✓	✓BUG	0.1
memctrl	bug2.sv	bug	34	✓BUG	kind-base	0.2	✓	✓BUG	0.1
pipeline_hazard	design.sv	safe	375	– UNKNOWN	bmc	351.9		– UNKNOWN	173.8
pipeline_hazard	bug1.sv	bug	375	✓BUG	kind-base	2.1	✓	✓BUG	0.2
priority_arbiter	design.sv	safe	23	✓SAFE	kind	0.7		✓SAFE	3.3
priority_arbiter	bug1.sv	bug	23	✓BUG	bmc	11.9	✓	✓BUG	0.5
round_robin_arbiter	design.sv	safe	15	✓SAFE	kind	0.1		✓SAFE	2.4
round_robin_arbiter	bug1.sv	bug	15	✓BUG	kind-base	0.1	✓	✓BUG	0.1
round_robin_arbiter	bug2.sv	bug	15	✓BUG	kind-base	0.0	✓	✓BUG	0.1
sync_fifo	design.sv	safe	15	✓SAFE	kind	0.1		✓SAFE	2.1
sync_fifo	bug1.sv	bug	15	✓BUG	kind-base	2.4	✓	✓BUG	0.5
sync_fifo	bug2.sv	bug	15	✓BUG	kind-base	0.1	✓	✓BUG	0.1
uart_tx	design.sv	safe	28	✓SAFE	kind	0.2		✓SAFE	1.6
uart_tx	bug1.sv	bug	28	✓BUG	bmc	17.1	✓	✓BUG	2.1
uart_tx	bug2.sv	bug	28	✓BUG	kind-base	1.6	✓	✓BUG	0.2
rv32i_soc	design.sv	safe	1878	✓SAFE	kind	4.7		✓SAFE	160.3
rv32i_soc	bug1.sv	bug	1878	✓BUG	kind-base	34.1	✓	✓BUG	1.4

10 Limitations and Future Work

The Lean artifacts are generated and pattern-matched against Sparkle's actual API but not compiler-verified in this environment (no network route to a Lean toolchain); wiring `lake build` into the agentic loop — with the LLM repairing elaboration errors — is the natural next step, as is discharging the emitted proof obligations with Lean tactics seeded by the engine's invariants. The engine's unbounded power currently ends at Houdini over template/LLM lemmas; IC3/PDR would extend it. Liveness is handled only through safety encodings (bounded-wait counters). LLM-dependent stages were exercised with the deterministic fallbacks in this evaluation because the sandbox blocks the OpenRouter endpoint; the code paths are complete and config-gated. Finally, mined-property precision depends on exploration depth; refuted conjectures are reported, never silently dropped, but a smarter sampler would raise the proposal hit-rate.

Table 3: Scalability: safe variants by state bits.

Benchmark	Bits	sparkle-fv	t(s)	smtbmc	t(s)
counter_wrap	10	SAFE	0.0	SAFE	0.9
axi_lite_slave	12	SAFE	0.1	SAFE	1.3
round_robin_arbiter	15	SAFE	0.1	SAFE	2.4
sync_fifo	15	SAFE	0.1	SAFE	2.1
lfsr	18	SAFE	0.0	SAFE	1.1
priority_arbiter	23	SAFE	0.7	SAFE	3.3
uart_tx	28	SAFE	0.2	SAFE	1.6
async_handshake	29	SAFE	0.8	SAFE	1.7
gray_counter	29	SAFE	0.1	SAFE	2.8
memctrl	34	UNKNOWN	300.0	UNKNOWN	151.0
alu32	85	SAFE	0.1	SAFE	19.0
pipeline_hazard	375	UNKNOWN	351.9	UNKNOWN	173.8
rv32i_soc	1878	SAFE	4.7	SAFE	160.3

11 Conclusion

sparkle-fv shows that the everyday-provers loop — formalize, search with provers, return inspectable artifacts, let AI accelerate but never vouch — is practical for hardware today: SystemVerilog in; Lean models, unbounded certificates, Verilator-confirmed bugs, and a DV-grade convergence dossier out. On a real Linux-booting SoC it proved the full architectural invariant set in about a minute and, in the process of writing properties, found two real bugs that a decade of simulation had not.

References

[1] Pramaana Labs. *The Age of Everyday Provers.* `https://pramaanalabs.ai/blog/the-age-of-everyday-provers`, 2026.

[2] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. *FME*, 2001.

[3] A. Niemetz, M. Preiner, C. Wolf, A. Biere. BTOR2, BtorMC and Boolector 3.0. *CAV*, 2018.

[4] C. Wolf. Yosys Open SYnthesis Suite. `https://yosyshq.net/yosys/`.

[5] M. Sheeran, S. Singh, G. Stålmarck. Checking safety properties using induction and a SAT-solver. *FMCAD*, 2000.